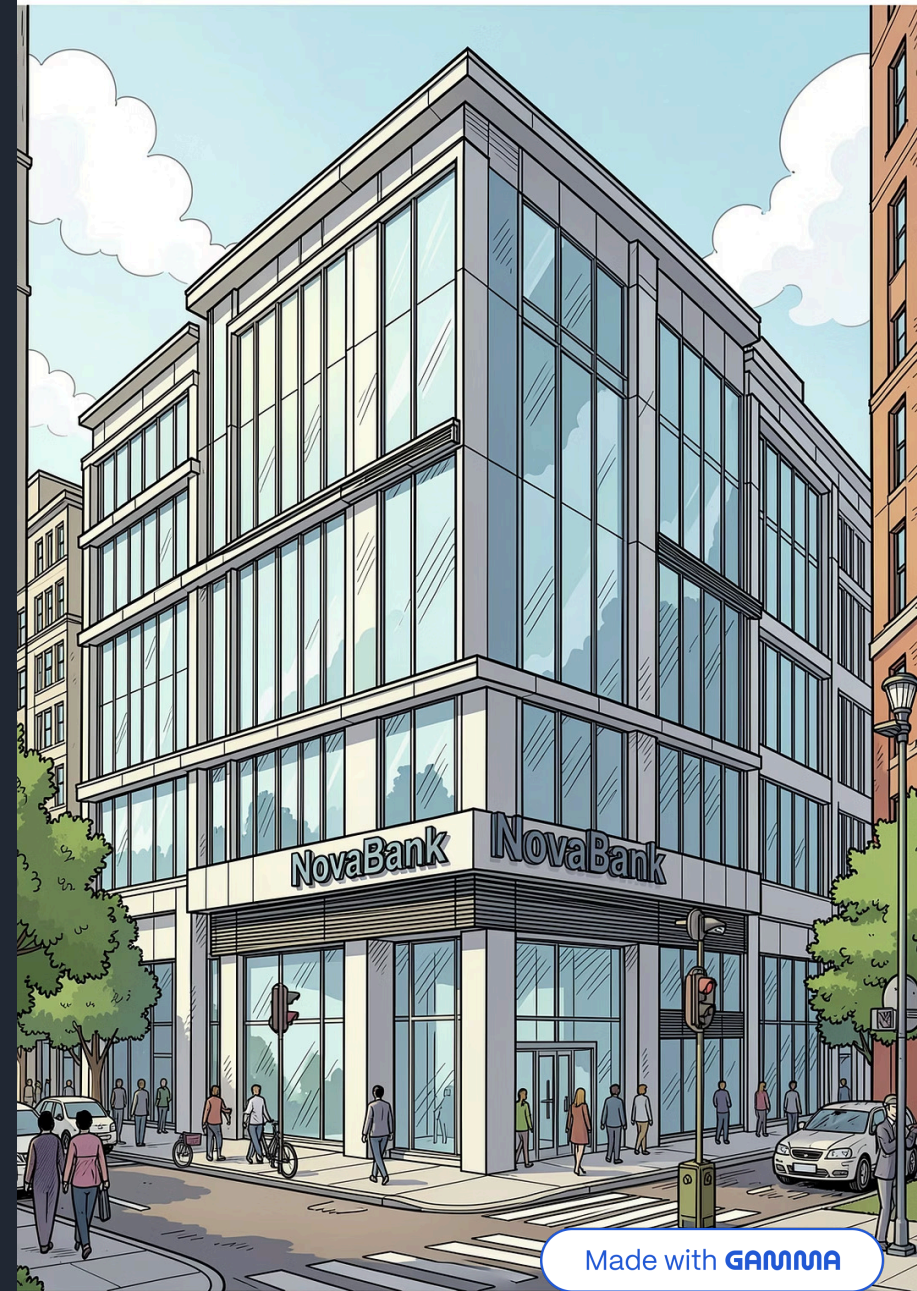


NovaBank: API REST Bancaria con Spring Boot

Trabajo de Fin de Grado · Evolución arquitectónica de una aplicación bancaria
hacia una API REST moderna

TFG · INFORMÁTICA

SPRING BOOT · REST · POSTGRESQL



Made with GAMMA

Contexto y Objetivos

El problema

Las aplicaciones monolíticas tradicionales dificultan el escalado de funcionalidades, el mantenimiento del código y la exposición de servicios a otros sistemas.

1. Encapsulación
2. Abstracción
3. Herencia
4. Polimorfismo

Objetivos del proyecto

- Transformar NovaBank en una API REST
- Persistencia en PostgreSQL con JPA
- Arquitectura multicapa y patrones de diseño
- Seguridad mediante JWT y Spring Security
- Documentación con Swagger/OpenAPI
- Base para migración a microservicios

Evolución del Proyecto

NovaBank ha recorrido un camino de maduración técnica en cuatro módulos progresivos.



Módulo 1

Aplicación de consola basada en POO



Módulo 2

Arquitectura por capas, JDBC y PostgreSQL



Módulo 3

Spring Boot, API REST, JWT y Swagger



Módulo 4

Migración a microservicios (futuro)

Arquitectura General



Separación de responsabilidades

Cada capa tiene una única responsabilidad, lo que facilita el mantenimiento, las pruebas unitarias y la escalabilidad del sistema.

Mantenibilidad

Escalabilidad

Testeabilidad

Modelo de Datos y Persistencia

```
--- INFORMACION DE CUENTA ---
```

```
Numero de cuenta: ES9121000000000000000002
```

```
Titular: Nadia Livensa
```

```
Saldo: 77,00 €
```

```
Fecha de creacion: 2026-03-26 11:01:55
```

Tecnologías

- **PostgreSQL** — base de datos relacional
- **Spring Data JPA + Hibernate** — ORM automático
- **Lombok** — reducción de código boilerplate

Entidades principales

Cliente

Datos personales y credenciales

Cuenta

Saldo y titularidad

Movimiento

Historial de operaciones

Diseño de la API REST

Recursos expuestos

Recurso	Endpoints
Autenticación	POST /login
Clientes	GET /api/clientes · POST
Cuentas	GET /api/cuentas · POST
Operaciones	POST /deposito · /retiro · /transferencia

Características de diseño

- Comunicación exclusivamente en **JSON**
- Uso de **DTOs** para el intercambio de datos
- Principios REST: verbos HTTP, URI claras
- Documentación interactiva con **Swagger/OpenAPI**

Seguridad Implementada

Autenticación sin estado mediante **Spring Security** y tokens **JWT**, protegiendo todos los endpoints de la API.

1

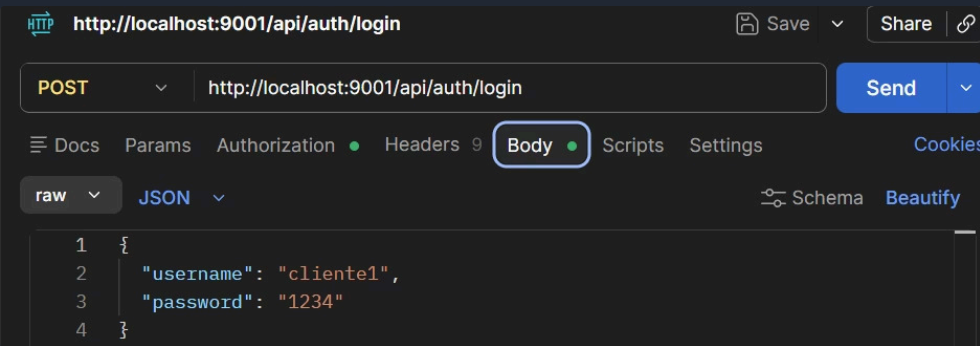
Inicio de sesión

El usuario envía credenciales al endpoint `/login`

2

Validación

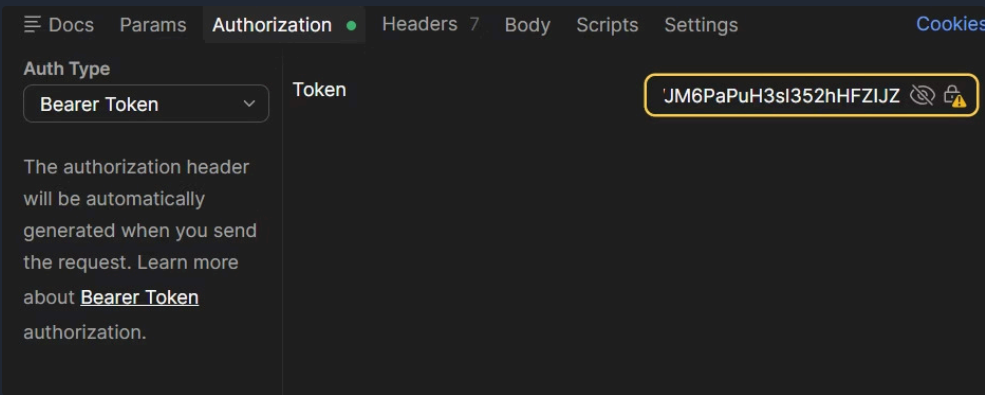
El sistema verifica usuario y contraseña contra la base de datos



3

Emisión del token

Se genera un JWT firmado con clave secreta y se devuelve al cliente

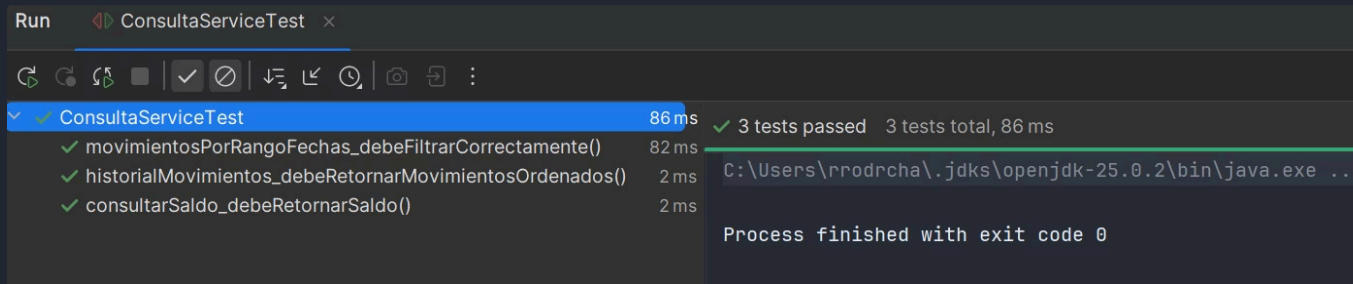


4

Acceso protegido

El cliente incluye el token en el header `Authorization` de cada petición

Pruebas y Validación



```
Run  ConsultaServiceTest x
✓ ConsultaServiceTest 86 ms ✓ 3 tests passed 3 tests total, 86 ms
  ✓ movimientosPorRangoFechas_debeFiltrarCorrectamente() 82 ms
  ✓ historialMovimientos_debeRetornarMovimientosOrdenados() 2 ms
  ✓ consultarSaldo_debeRetornarSaldo() 2 ms
C:\Users\rrodrcha\.jdk\openjdk-25.0.2\bin\java.exe ...
Process finished with exit code 0
```

Herramientas utilizadas

- **Postman** — pruebas manuales de endpoints
- **Swagger UI** — documentación y pruebas interactivas

Casos probados

- ✓ Todas las operaciones bancarias fueron validadas correctamente mediante pruebas funcionales.



Login



Clientes



Cuentas



Operaciones

Preparación para Microservicios



Descomposición planificada

El monolito se dividirá en servicios independientes, cada uno responsable de un dominio de negocio específico.

Spring Cloud

Ecosistema para microservicios

Eureka Server

Descubrimiento dinámico de servicios

API Gateway

Punto de entrada único y enrutamiento

Config Server

Configuración externalizada y centralizada

Conclusiones

✓ API REST funcional

De consola a API REST completa con Spring Boot

✓ Persistencia robusta

PostgreSQL + JPA con CRUD automático

✓ Seguridad JWT

Autenticación sin estado y endpoints protegidos

✓ Base para microservicios

Arquitectura preparada para escalar

Trabajo futuro

 **Rvbenrch** Merge pull request #18 fr

 .mvn/wrapper

 api-gateway

 auth-server

 cliente-service

 config-server

 cuenta-service

 eureka-server

 operacion-service

01

Completar la arquitectura de microservicios con Spring Cloud

02

Incorporar Docker para contenerización de servicios

03

Añadir monitorización, trazabilidad y despliegue en la nube

Dudas y preguntas

Contacto

Correo: rubenmanuel2003@gmail.com

[Rvbenrch/Microservicios_NovaBank at develop](#)

✓ Gracias por tu atención